

Aufgabe 09.02 - Lessons Learned im Team

Sozialform	Gruppenarbeit	Richtzeit	40 Minuten
Handlungsziele	Lessons Learned des Teams festhalten	Hilfsmittel	PC/Notebook, Teams, OneNote
Autor	Ramon Suter (bearbeitet durch Team MeetPoint)	Version	1.0
Projektleiter	Boris	Klasse	BMSD24a

Frage 1: Was haben wir während der Projektarbeit fachlich bzw. methodisch gelernt?

- **Fachlich – Vue 3 & TypeScript (Frontend):** Wir haben gelernt, wie man eine moderne, reaktive Single-Page-Application (SPA) mit Vue 3 (Composition API) und TypeScript aufbaut. Besonders das reaktive State-Management (ref, computed) und die Kommunikation mit externen APIs über Axios wurden praktisch gefestigt.
- **Fachlich – PostgREST & PostgreSQL (Backend):** Ein grosser Lerneffekt war der Einsatz von PostgREST als direkte RESTful API. Wir haben gelernt, wie man API-Endpunkte direkt aus dem SQL-Schema generiert und komplexe Berechtigungs- und Datenvalidierungen über Stored Procedures (PL/pgSQL RPCs) direkt in der Datenbank abbildet.
- **Fachlich – Docker Compose & Orchestrierung:** Wir haben gelernt, Multi-Container-Umgebungen (PostgreSQL + PostgREST) mit Docker Compose zu orchestrieren, inklusive Healthchecks zur Definition der exakten Boot-Reihenfolge der Services.
- **Methodisch – Git-Branching & Git-Workflows:** Die konsequente Arbeit mit Feature-Branches und Pull Requests hat uns gezeigt, wie sich Codeänderungen im Team sauber zusammenführen lassen, ohne dass es zu grossen Konflikten kommt.

Frage 2: Was waren für uns die grössten Herausforderungen und wie haben wir diese gemeistert?

- **Herausforderung 1 – API-Berechtigungsprüfung ohne JWT:** Da PostgREST alle Tabellenoperationen spiegelt, konnte standardmässig jeder Benutzer beliebige Daten manipulieren.
Lösung: Wir haben der standardmässigen Web-Rolle die DELETE- und UPDATE-Rechte entzogen und stattdessen sichere PL/pgSQL-Prozeduren geschrieben, die serverseitig die Berechtigung des aktuellen Users verifizieren, bevor sie die Transaktion ausführen.
- **Herausforderung 2 – Team-Ausfälle (Ressourcenengpass):** In der zweiten Projekthälfte fielen mehrere Teammitglieder krankheits- und verletzungsbedingt aus.
Lösung: Boris übernahm als Projektleiter die Aufgaben der fehlenden Mitglieder im Frontend

(Abstimmungslogik, Modals) und führte die Implementierung in Personalunion fort, um den Meilensteinplan zu halten.

- **Herausforderung 3 – Docker-Startreihenfolge:** PostgREST startete schneller als die PostgreSQL-Datenbank und stürzte ab, da die Datenbank noch keine Verbindungen zuließ.
Lösung: Wir haben in `docker-compose.yml` einen Datenbank-Healthcheck eingebaut und PostgREST erst starten lassen, wenn die Datenbank den Status `healthy` meldet.

Frage 3: Was haben wir als Team gut gemacht und würden wir das nächste Mal gleich machen?

- **Sorgfältige Vorplanung:** Die detaillierte Ausarbeitung des Entity-Relationship-Diagramms (ERD) und des SQL-Schemas in den ersten Wochen war die beste Entscheidung. Sie war das Fundament, auf dem der Projektleiter bei den Ausfällen nahtlos weiterarbeiten konnte.
- **Technologie-Entscheidung (PostgREST):** Der Verzicht auf ein manuell programmiertes Node.js/Python-Backend hat uns enorm viel Zeit erspart und Fehlerquellen minimiert.
- **Offene Kommunikation & Git-Struktur:** Das Team hat Fortschritte und Probleme stets transparent geteilt und Code sauber über Git versioniert.

Frage 4: Was haben wir als Team nicht perfekt gemacht und wie würden wir es das nächste Mal besser machen?

- **Stellvertreter-Konzept für Code-Aufgaben:** Die Aufgabenverteilung war sehr starr (eine Person macht nur DB, eine Person nur Frontend). Bei Ausfällen führte das zu Einarbeitungszeit.
Besser machen: Das nächste Mal werden wir ein gegenseitiges Stellvertreter-Konzept (Pair Programming oder Cross-Functional-Modell) einrichten, damit mindestens zwei Personen in jedem Code-Bereich handlungsfähig sind.
- **Integration von generiertem Code:** Die Integration von Code-Teilen, die via Prompts generiert wurden, war im Frontend oft zeitaufwendig, da die reaktiven State-Zusammenhänge in Vue komplex sind.
Besser machen: Wir würden früher gemeinsame Code-Reviews ansetzen und KI-generierten Code strenger isoliert testen.

Frage 5: Welche Erkenntnisse bzw. Erfahrungswerte nehmen wir aus dieser Projektarbeit mit?

- **Gute Planung zahlt sich doppelt aus:** Ein klares Konzept (DB-Schema, Stakeholder- und Risikoanalyse) macht die Umsetzung nicht nur einfacher, sondern dient bei unerwarteten Krisen als unverzichtbarer Kompass.
- **Standardisierte Umgebungen dank Docker:** Docker Compose ist ein riesiger Vorteil für die Teamarbeit, da alle Mitglieder mit exakt derselben DB- und API-Infrastruktur arbeiten.
- **Agilität in der Projektleitung:** Als Projektleiter muss man flexibel auf Ausfälle reagieren und bereit sein, Aufgaben neu zu priorisieren oder selbst operativ einzuspringen.